

목차 (Table of Contents)

체크리스트

[14:00] 워크플로우 규칙 설정

[14:20] 환경 설정

[14:50] 연구 방향성 수립 및 Alpamayo 조사

[15:15] Alpamayo 공식 모델 설치

[15:25] Alpamayo 추론 실행 (베이스라인 측정)

[15:37] 메모리 접근 패턴 프로파일링

[15:50] 실험 데이터 폴더 분리

[16:10] 4-bit 양자화 모델 프로파일링 및 비교

[16:30~20:30] 연구 방향 탐색 (자율 실행)

[21:00~] 세미나 자료 준비 & 보고서 재구성 (세션 2)

[21:00] 교수님 아이디어 분석 + 세미나 폴더

[21:30] Alpamayo 아키텍처 Q&A + 정리

[22:00] 보고서 섹션 병합 (구 4절+5절)

[22:30] FP16 이론적 최적값 & 구조적 한계 분석

[23:00] 보고서 섹션 분리 (1A/1B) + WSL2 오버헤드 + VRAM 시각화

[23:55~] 야간 자동 실험 (세션 3)

[23:55~02:00] 실험 1: 4-bit VRAM 제한별 성능 비교

[02:00~08:00] 실험 2: Demand Layering 구현 및 실험

작업 로그 — 2026-02-25

체크리스트

- [x] 멀티 에이전트 워크플로우 규칙 수립
 - [x] WSL → Windows Chrome 브라우저 연동
 - [x] gh CLI 설치 및 GitHub 인증
 - [x] GitHub 레포 생성 (SeungWoo3/alpamemyo, public)
 - [x] Slack GitHub 앱 구독 설정
 - [x] Alpamayo-R1-10B 모델 조사
 - [x] PC 스펙 확인
 - [x] Alpamayo 설치 방법 조사
 - [x] research.md 업데이트
 - [x] Alpamayo 공식 모델 설치 — Python 3.12 환경 구성
 - [x] Alpamayo 공식 모델 설치 — 레포 클론 및 의존성 설치
 - [x] Alpamayo 공식 모델 설치 — HuggingFace 모델 다운로드
 - [x] Alpamayo 설치/실행 가이드 문서 작성
 - [x] Alpamayo 공식 모델 추론 실행 (베이스라인 측정)
 - [x] 메모리 접근 패턴 프로파일링
 - [x] 실험 데이터 폴더 분리 (alpamayo ↔ experiments)
 - [x] 4-bit 양자화 모델 프로파일링 및 비교 시각화
 - [x] 알파마요 선행연구 리서치
 - [x] RTSS 2022 + 관련 연구 리서치
 - [x] 연구 방향 1: On-Demand Layering 분석 및 실험
 - [x] 연구 방향 2: 메모리 파이프라이닝 분석 및 실험
 - [x] 연구 방향 3: CPU-GPU 스왑 최적화 분석 및 실험
 - [x] 연구 방향 4: 창의적 접근 탐색 및 실험
 - [x] 통합 보고서 작성
-

[14:00] 워크플로우 규칙 설정

- **요청:** 멀티 에이전트 워크플로우 규칙 수립
- **상태:** 완료 ☑
- **수행 내용:** 4가지 핵심 규칙 정의 및 메모리에 저장
- **결과:** 작업 로그 기록 / 마스터 위임 전용 / 검증 에이전트 필수 / 자동 커밋 규칙 확정

[14:20] 환경 설정

- **요청:** WSL 브라우저 연동, GitHub 레포 생성, Slack 웹훅 연동
- **상태:** 완료 ☑
- **수행 내용:**
 - WSL → Windows Chrome 브라우저 연동 (~/.bashrc)
 - gh CLI 설치 및 인증
 - GitHub 레포 생성 (SeungWoo3/alpamemyo, public)
 - Slack GitHub 앱 구독 설정
- **결과:** 작업 완료 시 push → Slack 알림 파이프라인 구축
- **커밋:** 7d6908f, 3db349a

[14:50] 연구 방향성 수립 및 Alpamayo 조사

- **요청:** 연구 방향성 문서 작성, Alpamayo 모델 조사, PC 스펙 확인, 설치 방법 조사
- **상태:** 완료 ☑
- **수행 내용:**
 - Alpamayo-R1-10B 모델 조사 (NVIDIA 자율주행 VLA, 10B 파라미터, 24GB VRAM 요구)
 - PC 스펙 확인 (RTX 3080 Ti 12GB, i7-10700K, 16GB RAM, PCIe Gen3 x16)
 - 설치 방법 조사 (공식 레포 + 커뮤니티 4bit 양자화 모델 확인)
 - research.md 업데이트
- **결과:**
 - 이론 스왑 시간 ~0.75초 vs 실제 수 분 → 구조적 비효율 가설 확인
 - 4bit 양자화 모델(dwko/Alpamayo-R1-10B-4bit) 존재 → 12GB VRAM 실행 가능
 - 다음 단계: Alpamayo 설치 및 베이스라인 측정

[15:15] Alpamayo 공식 모델 설치

- 요청: Alpamayo-R1-10B 공식 모델 설치 (양자화 아닌 원본)
- 상태: 완료 ☑
- 수행 내용:
 - uv 0.10.6 + Python 3.12.12 설치
 - 레포 클론 → /home/seungwoo/workspace/alpamayo
 - ar1_venv 가상환경 생성 + 95개 패키지 설치
 - flash-attn 빌드 실패 → SDPA로 대체 (pyproject.toml, base_model.py 수정)
 - HuggingFace 토큰 로그인 + 모델 다운로드 (22.2GB, ~4분 30초)
 - 설치/실행 가이드 문서 작성 (docs/alpamayo-setup-guide.md)
 - 검증 에이전트로 문서 오류 6건 발견 → 수정 완료
- 결과:
 - 모델 캐시: ~/.cache/huggingface/hub/models-nvidia-Alpamayo-R1-10B (22.2GB)
 - 환경 검증: PyTorch 2.8.0, CUDA 사용 가능, alpamayo_r1 import 성공
 - 가이드 문서: docs/alpamayo-setup-guide.md (검증 완료)

[15:25] Alpamayo 추론 실행 (베이스라인 측정)

- 요청: 12GB VRAM에서 공식 모델(FP16) 추론 실행
- 상태: 완료 ☑
- 수행 내용:
 - test_inference.py 분석 → 데이터셋 Gated(403) 확인
 - 더미 입력 스크립트 작성 (test_dummy_inference.py)
 - 추론 실행 및 메모리 측정
- 결과:
 - 모델 로드: 54.2초 (20.65 GB 할당, 물리 VRAM 12GB 초과)
 - 추론: 280.5초 (Peak VRAM 21.52 GB)
 - CUDA Unified Memory로 자동 스와핑 발생 (OOM 없음)
 - 이론 스왑 시간(~0.6초) vs 실제(280초) → 구조적 비효율 확인

[15:37] 메모리 접근 패턴 프로파일링

- **요청:** 280초 추론의 시간 분해 및 VRAM 스와핑 패턴 분석
- **상태:** 완료 ☑
- **수행 내용:**
 - profile_memory.py 스크립트 작성 (5단계 시간 분해 + 0.5초 간격 메모리 로그)
 - 프로파일링 실행 (총 348.97초)
- **결과:**
 - Phase 3 (.to cuda): 56.86초 (16.3%) — 20.65GB를 12GB GPU에 올리면서 Unified Memory 스와핑
 - Phase 5 (VLM 추론): 273.79초 (78.5%) — 토큰당 1.07초, 정상 대비 20~30배 느림
 - Peak VRAM: 21.52GB (물리 12GB의 1.8배)
 - 메모리 로그: memory_profile.csv (704 샘플)
 - 핵심 원인: 매 토큰 생성마다 무작위 페이지 스왑 발생

[15:50] 실험 데이터 폴더 분리

- **요청:** 알파마요 소스코드와 실험 데이터/스크립트 폴더 구분
- **상태:** 완료 ☑
- **수행 내용:**
 - workspace/experiments/ 디렉토리 생성
 - test_dummy_inference.py, profile_memory.py, memory_profile.csv 이동
 - sys.path 수정 (experiments/에서 alpamayo 모듈 import 가능)
 - profile_memory.py CSV 출력 경로 상대경로로 수정
 - experiments/README.md 작성
- **결과:** alpamayo/ = NVlabs 원본 코드, experiments/ = 실험 스크립트/데이터로 분리 완료
- **커밋:** 8648f3d

[16:10] 4-bit 양자화 모델 프로파일링 및 비교

- **요청:** 12GB VRAM 이내에서 돌릴 수 있는 경량화 모델 동일 실험
- **상태:** 완료 ☑
- **수행 내용:**

- dwko/Alpamayo-R1-10B-4bit (BitsAndBytes fp4) 모델 조사
- bitsandbytes 패키지 설치
- profile_memory_4bit.py 작성 및 실행
- 시각화 3종 생성 (VRAM 시계열, Phase 분석, FP16 vs 4-bit 비교)
- **결과:**
- Peak VRAM: 8.87GB → 12GB 이내 완전 동작 (스와핑 없음)
- VLM 추론: 4.51초 (FP16 대비 55.8배 빠름, -98%)
- FP16의 273초 추론이 사실상 전부 메모리 스왑 오버헤드였음 확인
- 새 병목: 모델 로드 158초 (전체의 94%, 초회 HF 캐싱 포함)

[16:30~20:30] 연구 방향 탐색 (자율 실행)

- **요청:** RTSS 2022 Demand Layering 참고, 4개 연구 방향 분석/실험 + 선행연구 리서치 + 통합 보고서
 - **상태:** 완료 ☑
 - **수행 내용:**
 - 선행연구 리서치: Alpamayo 관련 논문 + GPU 메모리 최적화 25편
 - 방향 1 (On-Demand Layering): 레이어 구조 분석, device_map 실험 (추론 실패), 수동 오프로딩 분석
 - 방향 2 (메모리 파이프라이닝): 모듈별 순차 오프로딩, PCIe 대역폭 측정, CUDA 스트림 비동기 전송
 - 방향 3 (스왑 최적화): Unified Memory 분석, pinned vs pageable 비교, WSL2 오버헤드 측정
 - 방향 4 (창의적 접근): 7개 아이디어 탐색 (하이브리드 양자화, KV Cache, Speculative Decoding 등)
 - 통합 보고서 작성 (research/final-report.md)
 - **결과:**
 - device_map="auto" Alpamayo 비호환 (커스텀 토큰 처리)
 - WSL2 pageable D2H 4.5배 느림 → pinned memory 필수
 - 최적 전송 청크: 2-8MB
 - Top 3 권장: 하이브리드 양자화 > 디퓨전 스텝 축소 > 동적 해상도
 - 조합 시 예상: ~6.5GB Peak, ~3.2초 추론
-

[21:00~] 세미나 자료 준비 & 보고서 재구성 (세션 2)

[21:00] 교수님 아이디어 분석 + 세미나 폴더

- 요청: 교수님 2-stage 스와핑 아이디어 분석, 세미나 폴더 생성
- 상태: 완료 ☑
- 수행 내용:
- `seminar/` 폴더 생성
- `meeting-notes-01.md`: 교수님 아이디어(모듈 스와핑) vs `device_map="auto"` 비교 정리
- 핵심: 개념은 동일하나 Alpamayo는 `device_map` 비호환 → 커스텀 구현 필요
- 커밋: 12760e8

[21:30] Alpamayo 아키텍처 Q&A + 정리

- 요청: Expert, Flow Matching, CoT/CoC 등 아키텍처 학습
- 상태: 완료 ☑
- 수행 내용:
- Alpamayo 소스코드 분석으로 Expert, Flow Matching, KV Cache 전달 방식 규명
- `seminar/alpamayo-architecture.md`: 전체 파이프라인, 모듈 상세, 텐서 흐름, 메모리 분석
- `docs/info.md`: Expert, Flow Matching, CoT/CoC, 모델 네이밍 등 10개 항목 추가
- 커밋: cd37316

[22:00] 보고서 섹션 병합 (구 4절+5절)

- 요청: On-Demand Layering(4절)과 Memory Pipelining(5절) 같은 내용 → 합치기
- 상태: 완료 ☑
- 수행 내용:
- 4절+5절 통합: "On-Demand Layering & 메모리 파이프라이닝"
- 전체 섹션 재번호 (4개 방향 → 3개)
- A/B/C 시나리오 설명 추가
- 커밋: 7c3e263

[22:30] FP16 이론적 최적값 & 구조적 한계 분석

- 요청: 스왑 없는 환경의 이론적 추론 시간 계산
- 상태: 완료 ☑

- 수행 내용:
- GPU 대역폭(912 GB/s)으로부터 FP16 이론적 최적 추론 시간 산출: ~5.0s
- 연산 시간(0.44ms) vs 읽기 시간(18ms) → memory-bandwidth-bound 입증
- VRAM(912 GB/s) vs PCIe(8.5 GB/s) = 107배 격차 → 구조적 한계
- 최선의 스왑 최적화: 80-150s (이론 대비 16-30x)
- 커밋: ca47098, 221e88d

[23:00] 보고서 섹션 분리 (1A/1B) + WSL2 오버헤드 + VRAM 시각화

- 요청: 4절/5절을 합치지 말고 같은 주제의 다른 관점으로 분리, WSL2 오버헤드 기록, VRAM 점유 시각화
- 상태: 완료 ☑
- 수행 내용:
- “연구 방향 1(A): 메모리 스와핑 — 로딩 전략”과 “연구 방향 1(B): 전송 최적화 & 성능 한계”로 분리
- WSL2 오버헤드 섹션 추가 (보고서 5.3, 세미나 자료)
- VRAM 점유 비율 분석 & 시각화 3종 생성 (파이차트, 상세바, 시나리오 비교)
- KV Cache 2.4% (GQA 효과), 모델 파라미터 93%, Activation 4.6%
- docs/info.md 최신 분석 반영
- 생성 파일:
- seminar/figures/vram_breakdown.png
- seminar/figures/vram_detailed.png
- seminar/figures/vram_scenarios.png

[23:55~] 야간 자동 실험 (세션 3)

[23:55~02:00] 실험 1: 4-bit VRAM 제한별 성능 비교

- 요청: 4-bit 모델에서 VRAM 제한(12/10/8/6GB) 시 성능 비교
- 상태: 완료 ☑
- 수행 내용:
- research/05-vram-limit-comparison/run_vram_limit_test.py 작성 및 실행
- dummy 텐서로 VRAM 점유하여 가용 VRAM 제한
- 4개 조건에서 모델 로드/추론 시간, Peak VRAM 측정

- 시각화 3종 생성 (비교 차트, Performance Cliff, 요약 테이블)
- **결과:**
- 12GB: 4.79s (baseline)
- 10GB: 6.96s (1.45x) — 약간의 성능 저하
- 8GB: 179.46s (37.5x) — **급격한 성능 저하** (Performance Cliff)
- 6GB: 1506.88s (314.6x) — 25분 소요
- 모델 크기(8.87GB) 근처에서 cliff 발생 확인

[02:00~08:00] 실험 2: Demand Layering 구현 및 실험

- **요청:** FP16 Alpamayo에 레이어별 on-demand 로딩 적용
- **상태:** 완료 ☑
- **수행 내용:**
- 초기 구현: CPU-first 로딩 → OOM Kill 반복 (시스템 RAM 16GB < 모델 20GB)
- 5회 이상 OOM 실패 후 해결책 발견:
 - `torch.set_default_device("cuda")` 로 모델을 직접 CUDA에 생성
 - safetensors에서 shard별로 가중치를 CUDA에 직접 로드
 - 부분 오프로딩: VLM 30/36 레이어만 CPU, 6개 + Expert는 GPU 유지
- Forward hook으로 CPU↔GPU 전송 자동화
- 시각화 3종 생성
- **결과:**
- **추론 시간 116.51s** (Unified Memory 273.79s 대비 2.35x 빠름)
- **Peak VRAM 11.03GB** (12GB 물리 VRAM 이내!)
- VLM H2D 평균 64ms, D2H 평균 157ms
- D2H가 H2D보다 2.4배 느림 (WSL2 오버헤드 확인)
- **보고서:** [research/overnight-work-report.md](#)